



Lenguajes de Programación

IC4700

Proyecto 02

Analizador de eventos

Profesor:

Allan Rodríguez Davila

Estudiantes:

Heldyis Agüero Espinoza

Yeremi Calvo Porras

Fecha de entrega: 06/05/2026

I Semestre

Índice

1	Manual de Usuario	3
1.1	Descarga y compilación.....	3
1.2	Uso de los diferentes menús	4
1.3	Uso de funcionalidades directas	4
1.4	Funcionalidades secundarias en submenús	5
2	Prueba de funcionalidad.....	5
2.1	Transformación de eventos.....	5
2.1.1	Aplicar impuesto	5
2.1.2	Etiquetar objetos de alto valor	6
2.2	Análisis de datos	6
2.2.1	Monto total	6
2.2.2	Promedio anual	7
2.3	Análisis temporal.....	7
2.3.1	Máximo monto mensual y día más activo	7
2.3.2	Eventos máximos.....	8
2.3.3	Montos por intervalo	8
2.4	Búsqueda	9
2.5	Estadísticas	10
2.6	Salir.....	10
3	Descripción del problema.....	11
4	Diseño del programa	11
5	Librerías utilizadas	13
6	Análisis de resultados.....	13
7	Justificación de toma de decisiones	14
7.1	Funciones de orden superior utilizadas	14
7.2	Encadenado de las funciones.....	14
8	Bitácora	14

1 Manual de Usuario

1.1 Descarga y compilación

El programa se encuentra en el repositorio de GitHub que se brinda a continuación: https://github.com/HeldyisAE/Proyecto02_LenguajesDeProgramacion.git

Para su ejecución, se requiere la descarga del .zip del proyecto, una vez descargado y descomprimido se procede a abrir solo la carpeta llamada “Programa”. Una vez se haya abierto esta carpeta y VSCode o cualquier otro editor de confianza que utilice con el compilador designado para este proyecto lea y acepte el código, lo siguiente será abrir una terminal y ejecutar el comando “cabal build”, este comando compilará el proyecto y creará las carpetas de dependencias de cabal para la correcta ejecución del programa.

```
heldy@HeldyisAE MINGW64 ~/Desktop/Proyecto02_Lenguajes/Programa (develop)
$ cabal build
ama-tmp\Types\Event.o )
[ 4 of 19] Compiling Logic.Transformacion ( src\Logic\Transformacion.hs, dist-newstyle\build\x86_64-windows\ghc-9.6.7\Programa-0.1.0.0\x\Programa\build\Programa-tmp\Logic\Transformacion.o )
[ 5 of 19] Compiling Logic.Búsqueda ( src\Logic\Búsqueda.hs, dist-newstyle\build\x86_64-windows\ghc-9.6.7\Programa-0.1.0.0\x\Programa\build\Programa-tmp\Logic\Búsqueda.o )
[ 6 of 19] Compiling Helpers.OperacionesTemporales ( src\Helpers\OperacionesTemporales.hs, dist-newstyle\build\x86_64-windows\ghc-9.6.7\Programa-0.1.0.0\x\Programa\build\Programa-tmp\Helpers\OperacionesTemporales.o )
[ 7 of 19] Compiling Logic.Estadísticas ( src\Logic\Estadísticas.hs, dist-newstyle\build\x86_64-windows\ghc-9.6.7\Programa-0.1.0.0\x\Programa\build\Programa-tmp\Logic\Estadísticas.o )
[ 8 of 19] Compiling Helpers.OperacionesDatos ( src\Helpers\OperacionesDatos.hs, dist-newstyle\build\x86_64-windows\ghc-9.6.7\Programa-0.1.0.0\x\Programa\build\Programa-tmp\Helpers\OperacionesDatos.o )
[ 9 of 19] Compiling Logic.AnálisisDatos ( src\Logic\AnálisisDatos.hs, dist-newstyle\build\x86_64-windows\ghc-9.6.7\Programa-0.1.0.0\x\Programa\build\Programa-tmp\Logic\AnálisisDatos.o )
[10 of 19] Compiling Logic.AnálisisTemporal ( src\Logic\AnálisisTemporal.hs, dist-newstyle\build\x86_64-windows\ghc-9.6.7\Programa-0.1.0.0\x\Programa\build\Programa-tmp\Logic\AnálisisTemporal.o )
[11 of 19] Compiling Data.Registro ( src\Data\Registro.hs, dist-newstyle\build\x86_64-windows\ghc-9.6.7\Programa-0.1.0.0\x\Programa\build\Programa-tmp\Data\Registro.o )
[12 of 19] Compiling Data.GeneradorEventos ( src\Data\GeneradorEventos.hs, dist-newstyle\build\x86_64-windows\ghc-9.6.7\Programa-0.1.0.0\x\Programa\build\Programa-tmp\Data\GeneradorEventos.o )
[13 of 19] Compiling UI.MenuAnálisisDatos ( src\UI\MenuAnálisisDatos.hs, dist-newstyle\build\x86_64-windows\ghc-9.6.7\Programa-0.1.0.0\x\Programa\build\Programa-tmp\UI\MenuAnálisisDatos.o )
[14 of 19] Compiling UI.MenuAnálisisTemp ( src\UI\MenuAnálisisTemp.hs, dist-newstyle\build\x86_64-windows\ghc-9.6.7\Programa-0.1.0.0\x\Programa\build\Programa-tmp\UI\MenuAnálisisTemp.o )
[15 of 19] Compiling UI.MenuEstadísticas ( src\UI\MenuEstadísticas.hs, dist-newstyle\build\x86_64-windows\ghc-9.6.7\Programa-0.1.0.0\x\Programa\build\Programa-tmp\UI\MenuEstadísticas.o )
[16 of 19] Compiling UI.MenuTransformacion ( src\UI\MenuTransformacion.hs, dist-newstyle\build\x86_64-windows\ghc-9.6.7\Programa-0.1.0.0\x\Programa\build\Programa-tmp\UI\MenuTransformacion.o )
[17 of 19] Compiling UI.OpcionBúsqueda ( src\UI\OpcionBúsqueda.hs, dist-newstyle\build\x86_64-windows\ghc-9.6.7\Programa-0.1.0.0\x\Programa\build\Programa-tmp\UI\OpcionBúsqueda.o )
[18 of 19] Compiling UI.MenuPrincipal ( src\UI\MenuPrincipal.hs, dist-newstyle\build\x86_64-windows\ghc-9.6.7\Programa-0.1.0.0\x\Programa\build\Programa-tmp\UI\MenuPrincipal.o )
[19 of 19] Compiling Main ( app\Main.hs, dist-newstyle\build\x86_64-windows\ghc-9.6.7\Programa-0.1.0.0\x\Programa\build\Programa-tmp\Main.o )
[20 of 20] Linking dist-newstyle\build\x86_64-windows\ghc-9.6.7\Programa-0.1.0.0\x\Programa\build\Programa.exe
```

Se recomienda ejecutar de manera opcional posteriormente ejecutar el comando “clear” para limpiar la terminal y tener así una ejecución más limpia. Lo siguiente que el usuario debe hacer es simplemente ejecutar el comando “cabal run Programa” y esto empezará la ejecución.

```
heldy@HeldyisAE MINGW64 ~/Desktop/Proyecto02_Lenguajes/Programa (develop)
$ cabal run Programa
--- Menu Principal ---
1. Transformacion de eventos
2. Analisis de datos
3. Analisis temporal
4. Búsqueda
5. Estadísticas
0. Salir
>
```

1.2 Uso de los diferentes menús

El programa ofrece 4 diferentes menús, contabilizando el menú principal mostrado anteriormente. Para utilizar estos menús se requiere que el usuario ingrese el número que acompaña a la opción mostrada en el menú, si el usuario ingresa otro valor el programa le avisará de que ingresó una entrada inválida y se le volverá a ofrecer el mismo menú de nuevo. Este funcionamiento es el mismo para todos los submenús del sistema.

```
--- Transformación de eventos ---  
1. Aplicar impuesto a compras (13%)  
2. Etiquetar eventos de alto valor  
0. Volver
```

```
--- Analisis de datos ---  
1. Monto total (Importe total de todos los eventos)  
2. Promedio anual por categoría  
0. Volver
```

```
--- Analisis temporal ---  
1. Maximo monto mensual y dia más activo  
2. Ver evento más reciente y más antiguo  
3. Resumen de montos por intervalo  
0. Volver
```

1.3 Uso de funcionalidades directas

Como se menciona en la sección anterior, el programa cuenta con 4 submenús, pero como se puede ver en el menú principal este cuenta con 5 opciones en total, excluyendo la opción de salir. Para efectos de este proyecto denominamos funcionalidades directas a la opción de búsqueda y estadísticas, esto porque el usuario no verá ninguna opción como se muestran en los menús.

En el caso de la opción de búsqueda, al usuario se le solicitan 2 fechas, una de inicio y otra de fin para el intervalo de búsqueda.

```
--- Menu Principal ---  
1. Transformacion de eventos  
2. Analisis de datos  
3. Analisis temporal  
4. Busqueda  
5. Estadisticas  
0. Salir  
> 4  
  
Ingrese fecha inicio (dd-mm-aaaa):  
> 01-01-2026  
Ingrese fecha fin (dd-mm-aaaa):  
> 01-01-2027
```

Se espera que el usuario ingrese una fecha en el formato el formato válido, de lo contrario la opción no podrá ejecutarse y se volverán a solicitar las fechas.

En cuanto en la opción de estadísticas, se ejecuta tras seleccionar la opción, no se le solicita ninguna entrada al usuario.

1.4 Funcionalidades secundarias en submenús

En el programa hay algunas opciones, dentro de los submenús, que también requieren entradas del usuario.

En cuanto a la función para mostrar eventos por intervalo, esta requiere que el usuario ingrese un número, que representa el intervalo en días, para poder mostrar la información requerida.

```
--- Analisis temporal ---
1. Maximo monto mensual y dia más activo
2. Ver evento más reciente y más antiguo
3. Resumen de montos por intervalo
0. Volver
> 3

Ingrese el intervalo en días (1 - 727):
```

Como se puede ver en la impresión, el programa se informa al usuario la cantidad de días que hay desde el evento más antiguo en el sistema hasta el más nuevo, esto para proveerle información al usuario y sepa cuál es el intervalo máximo que puede ingresar.

2 Prueba de funcionalidad

2.1 Transformación de eventos

2.1.1 Aplicar impuesto

```
--- Transformación de eventos ---
1. Aplicar impuesto a compras (13%)
2. Etiquetar eventos de alto valor
0. Volver
> 1

[!] Impuesto(13%) aplicado a las compras.
```

2.1.2 Etiquetar objetos de alto valor

```
--- Transformación de eventos ---
1. Aplicar impuesto a compras (13%)
2. Etiquetar eventos de alto valor
0. Volver
> 2

--- Eventos de Alto Valor---

ID: 0
Categoria: compra
Valor: 31979.0 colones
Fecha: 14-09-2027
Cantidad: 23
Impuesto aplicado: 0.0
-----
```

2.2 Análisis de datos

2.2.1 Monto total

```
--- Analisis de datos ---
1. Monto total (Importe total de todos los eventos)
2. Promedio anual por categoría
0. Volver
> 1

    Desglose del total de importes
-----
Ganancias: 104270640.00 colones
Devoluciones: 19259072.00 colones
Total general: 85011616.00 colones
-----
```

2.2.2 Promedio anual

```
--- Analisis de datos ---
1. Monto total (Importe total de todos los eventos)
2. Promedio anual por categoría
0. Volver
> 2

    Promedio anual por categoría
-----
apartado (2026): 155515.63
apartado (2027): 198525.53
apartado (2028): 218886.22
compra (2026): 1219364.8
compra (2027): 837584.44
compra (2028): 1492275.4
devolucion (2026): -498937.72
devolucion (2027): -862048.8
devolucion (2028): -249627.0
seguimiento (2026): 38385.145
seguimiento (2027): 35246.0
seguimiento (2028): 16686.6
visualizacion (2026): 27665.295
visualizacion (2027): 35364.19
visualizacion (2028): 47212.223
-----
```

2.3 Análisis temporal

2.3.1 Máximo monto mensual y día más activo

```
--- Analisis temporal ---
1. Maximo monto mensual y dia más activo
2. Ver evento más reciente y más antiguo
3. Resumen de montos por intervalo
0. Volver
> 1

--- Mes(es) con mayor monto ---
    Febrero 2028 --- 10817735.00 Colones

--- Día(s) de la semana más activo ---
    Sabado --- 49 eventos
```

2.3.2 Eventos máximos

```
--- Analisis temporal ---
1. Maximo monto mensual y dia más activo
2. Ver evento más reciente y más antiguo
3. Resumen de montos por intervalo
0. Volver
> 2

--- Evento más antiguo ---

ID: 217
Categoria: visualizacion
Valor: 19416.0 colones
Fecha: 07-05-2026
Cantidad: 1
Impuesto aplicado: 0.0
-----

--- Evento más reciente ---

ID: 113
Categoria: apartado
Valor: 51717.0 colones
Fecha: 05-05-2028
Cantidad: 42
Impuesto aplicado: 0.0
-----
```

2.3.3 Montos por intervalo

```
--- Analisis temporal ---
1. Maximo monto mensual y dia más activo
2. Ver evento más reciente y más antiguo
3. Resumen de montos por intervalo
0. Volver
> 3

Ingrese el intervalo en días (1 - 729): 500
Intervalo: 500 días
-----

Del 07/05/2026 al 19/09/2027 | 180 eventos | 53136176.00 Colones
Del 19/09/2027 al 05/05/2028 | 114 eventos | 49353420.00 Colones
```


2.4 Búsqueda

```
--- Menu Principal ---
1. Transformacion de eventos
2. Analisis de datos
3. Analisis temporal
4. Busqueda
5. Estadisticas
0. Salir
> 4

Ingrese fecha inicio (dd-mm-aaaa):
> 05-05-2026
Ingrese fecha fin (dd-mm-aaaa):
> 08-08-2026

ID: 7
Categoria: visualizacion
Valor: 19899.0 colones
Fecha: 19-06-2026
Cantidad: 1
Impuesto aplicado: 0.0
-----

ID: 11
Categoria: compra
Valor: 83918.32 colones
Fecha: 15-06-2026
Cantidad: 22
Impuesto aplicado: 9654.319
-----
```

2.5 Estadísticas

```
=====
RESUMEN DE ESTADISTICAS
=====

Eventos por Categoria:
- apartado: 89
- compra: 106
- devolucion: 36
- seguimiento: 35
- visualizacion: 69

>>> EVENTO DE MAYOR VALOR <<<

ID: 11
Categoria: compra
Valor: 83918.32 colones
Fecha: 15-06-2026
Cantidad: 22
Impuesto aplicado: 9654.319
-----
>>> EVENTO DE MENOR VALOR <<<

ID: 195
Categoria: apartado
Valor: 756.0 colones
Fecha: 03-08-2027
Cantidad: 38
Impuesto aplicado: 0.0
-----
Dia mas activo: 23/10/2027 (3 eventos)
=====

¿Desea exportar el reporte a JSON?
1. Si
0. No
> 
```

2.6 Salir

```
--- Menu Principal ---
1. Transformacion de eventos
2. Analisis de datos
3. Analisis temporal
4. Busqueda
5. Estadisticas
0. Salir
> 0
Registro guardado: 346 eventos
Saliendo del programa...
```

Al salir del programa, se realiza la escritura de los eventos en un archivo .csv preestablecido, además el sistema informa del total de eventos escritos, que también representa la cantidad total de eventos que existen hasta ese momento.

3 Descripción del problema

El sistema consiste en una plataforma de monitoreo de actividad para comercio electrónico desarrollada en Haskell , la cual procesa eventos representados como datos algebraicos que incluyen identificador, categoría, valor y marca de tiempo. El núcleo del programa debe gestionar un flujo dinámico y potencialmente infinito de información , generando automáticamente entre 10 y 25 nuevos eventos tras cada interacción del usuario para simular un entorno de datos en tiempo real. A través de un menú interactivo, el sistema aplica operaciones del paradigma funcional para la transformación de datos (como el cálculo del 13% de impuesto), análisis estadístico, búsquedas por fechas y generación de reportes en formatos CSV o JSON. Además, aunque la información es inicialmente volátil en memoria , se implementa una capa de persistencia en medios físicos para mantener la continuidad de los registros entre sesiones, garantizando la integridad del sistema al evitar la duplicidad de identificadores y validando que todos los montos procesados sean superiores a cero.

4 Diseño del programa

El desarrollo del presente trabajo se llevó a cabo en Haskell en conjunto con el gestor cabal. Se planteó una estructura modular estándar del sistema para tener una mejor organización visual del código y de los módulos existentes en este.

```
PROGRAMA/
├── dist-newstyle/
├── src/
│   ├── Data/
│   │   ├── GeneradorEventos.hs
│   │   └── Registro.hs
│   ├── Helpers/
│   │   ├── HerramientasImpresion.hs
│   │   ├── OperacionesDatos.hs
│   │   ├── OperacionesFechas.hs
│   │   └── OperacionesTemporales.hs
│   ├── Logic/
│   │   ├── AnalisisDatos.hs
│   │   ├── AnalisisTemporal.hs
│   │   ├── Busqueda.hs
│   │   ├── Estadisticas.hs
│   │   └── Transformacion.hs
│   ├── Storage/
│   │   └── Registro.csv
│   ├── Types/
│   │   └── event.hs
│   └── UI/
│       ├── MenuAnalisisDatos.hs
│       ├── MenuAnalisisTemp.hs
│       ├── MenuEstadisticas.hs
│       ├── MenuPrincipal.hs
│       ├── MenuTransformacion.hs
│       └── OpcionBusqueda.hs
├── CHANGELOG.md
├── LICENSE
├── Programa.cabal
└── reporte.json
```

Para la generación de eventos se implementó un módulo dedicado a este propósito, los objetos de tipo evento tienen la siguiente estructura:

```
data Event = Event
  { idEvent  :: Int
  , category :: String
  , value    :: Float
  , timestamp :: Int
  , amount   :: Int
  , tax      :: Float
  , tag      :: Bool
  }
```

Como se puede notar, además de los atributos mínimos especificados en el enunciado del proyecto, se añadieron 3 atributos más. Uno representa la cantidad de objetos por evento, es una abstracción de objetos adquiridos o los eventos con que interactúa un usuario. Este valor se implementó para añadirle realismo y complejidad al calculo de importes de los eventos.

Los valores tax y tag tienen el propósito de proporcionar un mejor control de la información por evento. tax nos permite saber si un evento de tipo compra ya fue transformado para aplicarle el impuesto y el total que le corresponde. Tag es un valor booleano que nos permite determinar cuáles son los últimos eventos que fueron marcados como eventos de alto valor.

Para la persistencia de la información, se adoptó una estrategia sencilla: Leer el archivo de persistencia al inicio de la ejecución, cargar los eventos y al finalizar cada ejecución reescribir el archivo con los nuevos eventos y todas las modificaciones que se hayan aplicado a los antiguos.

5 Librerías utilizadas

A continuación, se enumeran las librerías utilizadas:

1. **Random:** utilizada para simular la creación de eventos con datos aleatorios de manera que se asignan categorías, montos y determinando cantidades.
2. **Time:** encargada de la gestión del tiempo en el sistema además de la asignación de las fechas, también se asegura que las fechas ingresadas por los usuarios sean realmente validas en un calendario.
3. **Data.List:** importante para la manipulación de datos, principalmente utilizando funciones de groupBy, sortBy,sum y filter. De esta manera se mantiene un control y orden en la manera en que se manejan los datos.
4. **Data.Ord:** Con esta se manejan lo que vienen a ser ordenamientos se utilizan los datos de los eventos por ejemplo costos de estos y se utilizaría para sacar el máximo costo.
5. **Data.Function:** normalmente para combinar funciones de comparación, por ejemplo, agrupando eventos con la misma categoría ayudando al orden del código
6. **Data.Char:** utilizado principalmente en Helpers.OperacionesFechas y lo que hace es validar que los valores ingresados por los usuarios en fechas sean números y no letras para evitar fallos.
7. **System.IO:** Control del flujo de entrada/salida y limpieza del buffer de la terminal.
8. **Text.Printf:** Para la interpretación de cadenas y formateo de datos numéricos, de manera que mostrar datos que son por lo general float sea más sencillo

6 Análisis de resultados

Objetivo	Alcanzado	No alcanzado
Implementación del modelo de datos	X	
Generación dinámica de eventos	X	
Transformación	X	
Análisis de datos	X	
Análisis temporal	X	
Búsqueda	X	
Estadísticas	X	
Persistencia de datos	X	

7 Justificación de toma de decisiones

7.1 Funciones de orden superior utilizadas

A continuación, se procede con la enumeración de las funciones de orden superior utilizadas en el programa y una breve justificación de su uso en este.

- **map:** Se utiliza principalmente para recorrer elementos evento y obtener los valores de alguno de sus parámetros, también se utiliza en alguna otra función para simplemente recorrer listas.
- **filter:** El uso más destacado de esta función en el sistema es cuando se le utiliza para filtrar y buscar fechas específicas dentro de la lista de eventos. En otras funciones también se utiliza de forma secundaria para descartar o apartar alguna información.
- **foldr:** Se utiliza solamente en el módulo registro para recorrer una cadena de texto y manejar mejor la impresión en el csv con las (,) de separación.
- **all:** Se utiliza en el módulo de operaciones con fechas para verificar que todos los elementos de una string con formato de fecha sea un dígito numérico.
- **on:** Se usa en el proyecto, en la función agruparPor, para aplicar la función f a ambos argumentos

7.2 Encadenado de las funciones

Como se mencionó previamente en este documento, se planteó una estructura de organización modular. Esto nos permitió utilizar archivos que contienen funciones auxiliares y reutilizar funciones dentro de otras cuando es necesario. Para esto se siguió la mayoría con el formato de importación no implícita, sin embargo, durante el desarrollo del proyecto comprendimos que la importación implícita reduce el uso de los recursos durante la ejecución del programa, esto por esto que el código se encontrarán algunas importaciones implícitas de módulos que en otros módulos se importan de manera no implícita.

En cuanto a la comunicación entre funciones, dado que en Haskell no se pueden utilizar identificadores o funciones globales, se optó por una estrategia más secuencial y recursiva para manipular los datos entre las funciones. El núcleo del programa es el envío de los eventos entre las funciones, es por esto que la función `menuPrincipal` recibe una lista de eventos. Esta lista se obtiene de 2 posibles puntos distintos: Si el sistema no tiene eventos registrados previamente, se llama a la función encargada de generar funciones al inicio de la ejecución del menú principal, la generación de eventos devuelve una lista y esta es la que se envía y comparte entre todas las demás funciones. Si el sistema tiene eventos ya registrados, el sistema los lee antes de llamar al menú principal y se los pasa como parámetro para que estos sean añadidos a la generación dinámica de eventos.

8 Bitácora

Se adjunta el enlace al repositorio de Github donde fue desarrollado este trabajo:

https://github.com/HeldyisAE/Proyecto02_LenguajesDeProgramacion.git